



PROGRAMAÇÃO ORIENTADA À OBJETOS

AULA 02



AGENDA

01

ASPECTOS FUNDAMENTAIS DE JAVA

02

ESTRUTURAS CONDICIONAIS E DE EXCEÇÃO



01

ASPECTOS FUNDAMENTAIS DE JAVA



01 Tipos de Dados



- Um caminhão pode transportar diferentes objetos, cada um deles pode exigir uma embalagem diferente para ser armazenado. Em função disso, cada objeto pode exigir um espaço físico diferente no compartimento de carga;
- É importante para que os produtos permaneçam bem acomodados no compartimento de carga para que cheguem ao destino da mesma maneira como iniciaram a viagem, isto é, mantendo suas características sem sofrerem nenhum dano;
- Para que o exposto seja verdade, é essencial utilizar a embalagem correta para cada tipo de produto;

01 Tipos de Dados



- De forma semelhante, em computação existem diferentes tipos de dados (tipos numéricos, caracteres, textuais etc.), e cada um exige uma "embalagem" adequada para seu armazenamento;
- Considere que seja necessário armazenar na memória do computador os seguintes dados de uma pessoa: nome, sexo e idade. O nome contém uma sequência de caracteres, o sexo contém uma única letra (M ou F) e a idade um valor numérico;
- Para armazenar esses dados, um programa deve definir variáveis, isto é, identificadores de um determinado tipo de dado;



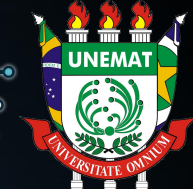
- Os tipos de dados em Java são portáveis entre todas as plataformas de computadores que suportam essa linguagem;
- Na maioria das linguagens, quando um tipo de dado é utilizado, por exemplo, um tipo inteiro, pode ser que para uma determinada plataforma esse número seja armazenado com 16 *bits* e em outra com 32 *bits*;
- Em Java isso não ocorre independentemente da plataforma utilizada. Este e muitos outros recursos de portabilidade de Java permitem que os programadores escrevam programas sem conhecer a plataforma de computador em que eles serão executados;



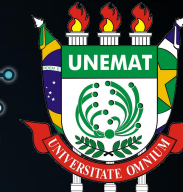
Tipo	Quantidade de bits	Valores
char	16	'\u0000' a '\uFFFF'
byte	8	-128 a + 127
int	32	-2.147.483.648 a +2.147.483.647
short	16	-32.768 a + 32.767
long	64	-9.223.372.036.854.775.808 a +9.223.372.036.854.775.807
float	32	-3.40292347E+38 a +3.40292347E+38
double	64	-1.79769313486231570E+308 a +1.79769313486231570E+308
boolean	8	true ou false



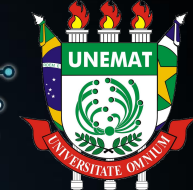
- Quando os recursos de hardware da máquina eram mais limitados, existia a preocupação de reduzir ao máximo a quantidade de memória utilizada por um programa; não que isso não ocorra hoje, mas antigamente era prioritário;
- Analisando a tabela anterior, um programador chegaria à conclusão de que para armazenar a idade de uma pessoa deveria ser utilizado o tipo *byte*, pois possui uma faixa de valores suficiente para armazenar a idade que uma pessoa normal pode atingir, dispensando outros tipos numéricos que ocupariam mais espaço na memória;



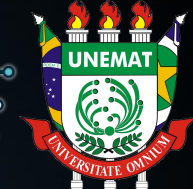
- Uma variável ou constante é um tipo de identificador cujo nome, escolhido pelo programador, é associado a um valor pertencente a um certo tipo de dado;
- Um identificador é a localização da memória capaz de armazenar o valor de um certo tipo, para o qual se dá um nome (o nome da variável, constante, objeto etc.). Desta forma, todo identificador possui um nome, um tipo e um conteúdo;
- Os identificadores não podem utilizar as palavras reservadas da linguagem Java, assim como ocorre com outras linguagens de programação;



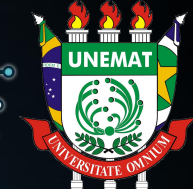
abstract	default	goto *	Package	this
assert ***	do	if	Private	throw
boolean	double	implements	protected	throws
break	else	import	public	transient
byte	enum ****	instanceof	Return	try
case	extends	int	Short	true
catch	false	interface	static	void
char	final	long	strictfp **	volatile
class	finally	native	Super	while
const *	float	new	Switch	
continue	for	null	Synchronized	
*	não usado			
**	adicionado à versão 1.2			
***	adicionado à versão 1.4			
****	adicionado à versão 1.5			



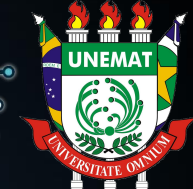
- A linguagem Java exige que todos os identificadores tenham um tipo de dado definido antes de serem utilizados no programa;
- Ou seja, eles devem ser obrigatoriamente declarados, independentemente do ponto do programa, seja no início, no meio ou no final, desde que antes de sua utilização no programa;
- Essa característica difere da maioria das linguagens de programação. Pascal, por exemplo, possui uma área exclusiva para a declaração das variáveis. Python, por exemplo, não exige a declaração do tipo de dado;



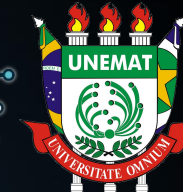
- Quando as variáveis são declaradas, a linguagem Java atribui a elas valores padrão, a menos que especificado de maneira contrária pelo programador;
- Atribui-se a todas as variáveis dos tipos *char*, *byte*, *short*, *int*, *long*, *float* e *double* o valor 0 por *default*. Já às variáveis do tipo *boolean*, por *default*, atribui-se *false*;
- Entretanto, dependendo do ponto do programa em que a variável é utilizada, torna-se obrigatória sua inicialização com algum valor, mesmo que a linguagem possua esses valores padrão;



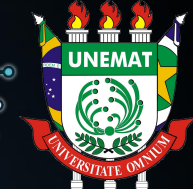
- As variáveis também possuem sensibilidade, isto é, ao declarar uma variável com o nome **dolar**, ela deve ser utilizada sempre da mesma forma;
- Não pode ser usada como **Dolar**, **DOLAR**, **dOlar** ou qualquer outra variação, apenas com todas as letras minúsculas, como realizado em sua declaração;
- Os nomes de variáveis devem começar com letra, caractere de sublinhado (**_**) ou cifrão (**\$**). Não é permitido iniciar o nome de uma variável com número;



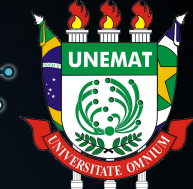
- Por convenção, a linguagem Java utiliza o seguinte padrão para nomear as variáveis (e outros indentificadores):
 - Quando o nome da variável for composto apenas por um caractere ou palavra, os caracteres devem ser minúsculos;
 - Quando o nome da variável tiver mais de uma palavra, a primeira letra, da segunda palavra em diante, deve ser maiúscula. Todos os outros caracteres devem ser minúsculos;
 - Exemplos: a, a1, \$preco, nome, valorVenda, codigoFornecedor;



```
1 class TiposPrimitivos {
2
3     public static void main (String args[]) {
4
5         char sexo = 'm';
6         byte idade = 89;
7         short codigo = 256;
8         float nota = 9.4f;
9         int alunos = 100, classes = 10;
10        long habitantes = 9050100;
11        double dolar = 2.62;
12        boolean alternativa = false;
13
14        System.out.println("sexo: "+sexo+" - idade: "+idade+" - codigo: "+codigo);
15        System.out.println("nota: "+nota+" - alunos: "+alunos+" - classes: "+classes);
16        System.out.println("habitantes: "+habitantes+" - dolar: "+dolar+" alternativa: "+alternativa);
17    }
18
19 }
```



- Uma constante é um tipo de variável que não pode alterar seu conteúdo depois de ter sido inicializado, ou seja, o conteúdo permanece o mesmo durante toda a execução do programa;
- Na realidade não existem constantes em Java, o que existe é um tipo de variável com comportamento semelhante a uma constante de outras linguagens;
- Em java, essa variável é definida como **final**. Essas constantes são usadas para armazenar valores fixos, geralmente definidos no início de uma classe;

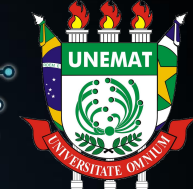


- Em java, padronizou-se identificar as variáveis do tipo **final** com todas as letras maiúsculas, e quando existe mais de uma palavra elas são separadas pelo caractere de *underline* (_);
- Exemplos de valores constantes são:
 - o valor de PI na matemática (3.14);
 - o valor da aceleração na física (9.81 m/s²);
 - a quantidade de milissegundos existentes em um segundo (1000);
 - etc;

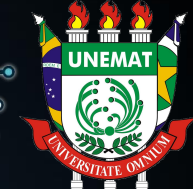


- Caso um segundo valor seja atribuído a uma variável **final** no decorrer da classe, o compilador gera uma mensagem de erro;
- Não é obrigatório inicializar o conteúdo de uma variável **final** no momento de sua declaração;

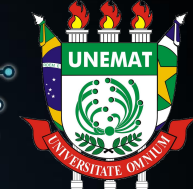
```
final double PI = 3.14;  
final int MILISEGUNDOS_POR_SEGUNDO = 1000;  
final long MILISEGUNDOS_POR_DIA = 24 * 60 * 60 * 1000;  
final int MILIMETROS_POR_CENTIMETRO;  
.  
.  
.  
MILIMETROS_POR_CENTIMETRO = 10;
```



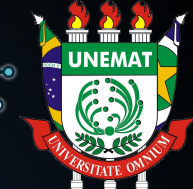
```
1  class Constantes {
2
3      public static void main (String args[]) {
4
5          final double PI = 3.14;
6          final int MILISEGUNDOS_POR_SEGUNDO = 1000;
7          final long MILISEGUNDOS_POR_DIA = 24 * 60 * 60 * 1000;
8          final int MILIMETROS_POR_CENTIMETRO;
9
10         MILIMETROS_POR_CENTIMETRO = 10;
11
12         System.out.println("PI: "+PI);
13         System.out.println("MILISEGUNDOS_POR_SEGUNDO: "+MILISEGUNDOS_POR_SEGUNDO);
14         System.out.println("MILISEGUNDOS_POR_DIA: "+MILISEGUNDOS_POR_DIA);
15         System.out.println("MILIMETROS_POR_CENTIMETRO: "+MILIMETROS_POR_CENTIMETRO);
16
17         MILISEGUNDOS_POR_SEGUNDO = 50;
18     }
19
20 }
```



- Os comentários são linhas adicionadas ao programa que servem para facilitar seu entendimento por parte do programador, ou ainda por uma outra pessoa que o consulte;
- Essas linhas não afetam o programa em si, pois não são consideradas parte do código;
- O Java aceita três tipos de comentário:
 - de uma linha;
 - de múltiplas linhas;
 - e de documentação;



- Para inserir comentários de linha única utilizam-se `//` (duas barras) em qualquer local do programa e tudo o que estiver escrito depois desse sinal e na mesma linha será considerado um comentário;
- Para inserir comentários que envolvam várias linhas, utiliza-se `/*` (barra asterisco) para demarcar o início e `*/` (asterisco barra) para o final;
- O comentário de documentação do programa, por meio de um utilitário (javadoc) fornecido pela Oracle junto com o JDK, inicia-se com `/**` e é encerrado com `*/`;

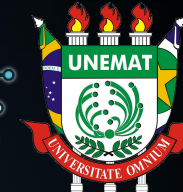


- Repliquem o exemplo abaixo:

```
1  /** Exemplo de Comentários:
2  Essa classe demonstra a utilização de variáveis em uma classe
3  em Java. São declaradas variáveis do tipo int e double.
4  O exemplo também demonstra como imprimir o conteúdo das
5  variáveis na tela.
6  */
7
8  class Comentarios {
9      public static void main (String args[]) {
10         //declaracao de variáveis do tipo inteiro
11         int matricula = 1089, idade = 20;
12         double dolar = 2.62; //declaracao de variável do tipo double
13         /* As linhas seguintes enviam o conteúdo das
14         variáveis para a tela */
15         System.out.println("Matricula: "+matricula+" - idade: "+idade+" - dolar: "+dolar);
16     }
17
18 }
```



Função	Sinal	Exemplo
Adição	+	$x + y$
Subtração	-	$x - y$
Multiplicação	*	$x * y$
Divisão	/	x / y
Resto da divisão inteira	%	$x \% y$
Sinal negativo	-	$-x$
Sinal positivo	+	$+x$
Incremento unitário	++	$++x$ OU $x++$
Decremento unitário	--	$--x$ OU $x--$



- Repliquem o exemplo abaixo:

```
1  class OperadoresAritmeticos {
2      public static void main (String args[]) {
3          //declaracao e inicialização de variáveis
4          int x = 10, y = 3;
5          //varias operações com variaveis
6          System.out.println("X = " + x);
7          System.out.println("Y = " + y);
8          System.out.println("-X = " + (-x));
9          System.out.println("-Y = " + (-y));
10         System.out.println("X/Y = " + (x/y));
11         System.out.println("Resto de X por Y = " + (x % y));
12         System.out.println("Inteiro de X por Y = " + (int) (x/y));
13         System.out.println("X + 1 = " + (++x));
14     }
15 }
16 }
```

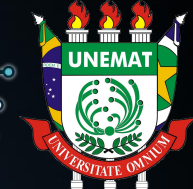


Função	Caractere(s) utilizado(s)	Exemplo
Igual	<code>==</code>	<code>x == y</code>
Diferente	<code>!=</code>	<code>x != y</code>
Maior que	<code>></code>	<code>x > y</code>
Maior ou igual a	<code>>=</code>	<code>x >= y</code>
Menor que	<code><</code>	<code>x < y</code>
Menor ou igual a	<code><=</code>	<code>x <= y</code>



Função	Caractere(s) utilizado(s)	Exemplo
E lógico ou AND	&&	x && y
Ou lógico ou OR		x y
Negação ou NOT	!	!x

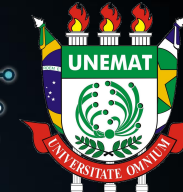
Sequência	Caractere especial
\b	Espaço
\f	Form feed
\n	Nova linha
\r	Retorno
\t	Tabulação
\"	Aspas duplas
\'	Aspas simples
\\	Barra invertida



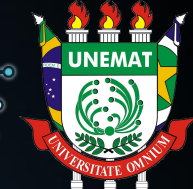
Sequência	Caractere especial
\b	Espaço
\f	Form feed
\n	Nova linha
\r	Retorno
\t	Tabulação
\"	Aspas duplas
\'	Aspas simples
\\	Barra invertida



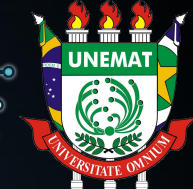
Supondo a variável x	Converter em	y recebe o valor convertido
int x = 10	float	float y = (float) x
int x = 10	double	double y = (double) x
float x = 10.5	int	int y = (int) x
String x = "10"	int	int y = Integer.parseInt(x)
String x = "20.54"	float	float y = Float.parseFloat(x)
String x = "20.54"	double	double y = Double.parseDouble(x)
String x = "Java"	Vetor de bytes	byte b[] = x.getBytes()
int x = 10	String	String y = String.valueOf(x)
float x = 10.35	String	String y = String.valueOf(x)
double x = 254.34	String	String y = String.valueOf(x)
byte x[] – (x é um vetor de bytes)	String	String y = new String(x)



```
1  class ConversaoTipos {
2      public static void main (String args[]) {
3          //declaracao e inicialização de variáveis
4          String s1 = "10";
5          //conversao de String para int
6          int v = Integer.parseInt(s1);
7          //conversao de String para Float
8          float x = Float.parseFloat(s1);
9          //conversao de String para double
10         double y = Double.parseDouble(s1);
11         //declaracao de variavel w com parte inteira de x
12         int w = (int) x;
13         //declaracao de variavel z com parte inteira de y
14         int z = (int) y;
15         //declaracao de variavel s2 com conversao de tipo
16         String s2 = String.valueOf(v);
17         //exibição em tela de soma de valores de variaveis
18         System.out.println(s2 + w + z);
19     }
20 }
21 }
```



- Uma aplicação provavelmente não receberá dados pelo *prompt* durante a execução. O mais comum é receber os dados do usuário por meio do teclado;
- Exemplos com quatro classes diferentes que permitem realizar esse processo:
 - `DataInputStream`;
 - `BufferedReader`;
 - `Scanner`;
 - e `JOptionPane`.



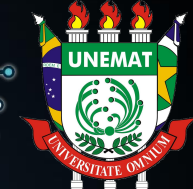
- **DataInputStream;**

```
1 //import da biblioteca I/O
2 import java.io.*;
3 class EntradaComDataInputStream {
4     public static void main (String args[]) {
5         //declaracao de variável
6         String s;
7         float largura, comprimento, area, perimetro;
8         DataInputStream dado;
9         //bloco de execução com tratamento de exceções
10        try{
11            //entrada de dados
12            System.out.println("Entre com o comprimento: ");
13            dado = new DataInputStream(System.in);
14            s = dado.readLine();
15            //conversao de tipo de dado
16            comprimento = Float.parseFloat(s);
17            //entrada de dados
18            System.out.println("Entre com a largura: ");
19            dado = new DataInputStream(System.in);
20            s = dado.readLine();
21            //conversao de tipo de dado
22            largura = Float.parseFloat(s);
23            //uso de operadores aritméticos
24            area = comprimento * largura;
25            perimetro = comprimento * 2 + largura * 2;
26            //Exibição de valores em tela
27            System.out.println("Área do retângulo: "+ area);
28            System.out.println("Perimetro do retângulo: "+ perimetro);
29        } catch (IOException erro) {
30            //Bloco acionado em caso de erro de I/O
31            System.out.println("Houve erro na entrada de dados: "+ erro.toString());
32        } catch (NumberFormatException erro){
33            //Bloco acionado em caso de erro de conversão de tipo
34            System.out.println("Houve erro na conversão, digite apenas "
35                + "caracteres numéricos: "+ erro.toString());
36        }
37    }
38 }
```



- **BufferedReader;**

```
1 //import da biblioteca I/O
2 import java.io.*;
3 class EntradaComBufferedReader {
4     public static void main (String args[]) {
5         //declaracao de variável
6         String s;
7         float largura, comprimento, area, perimetro;
8         BufferedReader dado;
9         //bloco de execução com tratamento de exceções
10        try{
11            //entrada de dados
12            System.out.println("Entre com o comprimento: ");
13            dado = new BufferedReader(new InputStreamReader(System.in));
14            s = dado.readLine();
15            //conversao de tipo de dado
16            comprimento = Float.parseFloat(s);
17            //entrada de dados
18            System.out.println("Entre com a largura: ");
19            dado = new BufferedReader(new InputStreamReader(System.in));
20            s = dado.readLine();
21            //conversao de tipo de dado
22            largura = Float.parseFloat(s);
23            //uso de operadores aritméticos
24            area = comprimento * largura;
25            perimetro = comprimento * 2 + largura * 2;
26            //Exibição de valores em tela
27            System.out.println("Área do retângulo: "+ area);
28            System.out.println("Perímetro do retângulo: "+ perimetro);
29        } catch (IOException erro) {
30            //Bloco acionado em caso de erro de I/O
31            System.out.println("Houve erro na entrada de dados: "+ erro.toString());
32        } catch (NumberFormatException erro){
33            //Bloco acionado em caso de erro de conversão de tipo
34            System.out.println("Houve erro na conversão, digite apenas "
35                + "caracteres numéricos: "+ erro.toString());
36        }
37    }
38 }
```



- **Scanner**: possui métodos específicos para leitura de diferentes tipos de dados.
- Observem as linhas 15 e 21. Outros exemplos de alguns métodos similares para leitura de valores numéricos são: `nextByte`, `nextShort`, `nextInt`, `nextLong`, `NextDouble`, entre outros;

```
1 //import da biblioteca Util
2 import java.util.*;
3 class EntradaComScanner {
4     public static void main (String args[]) {
5         //declaracao de variável
6         float largura, comprimento, area, perimetro;
7         Scanner sc;
8         //bloco de execução com tratamento de exceções
9         try{
10             //entrada de dados
11             System.out.println("Entre com o comprimento: ");
12             sc = new Scanner(System.in);
13             //conversao de tipo de dado
14             comprimento = sc.nextFloat();
15
16             //entrada de dados
17             System.out.println("Entre com a largura: ");
18             sc = new Scanner(System.in);
19             //conversao de tipo de dado
20             largura = sc.nextFloat();
21             //uso de operadores aritméticos
22
23             area = comprimento * largura;
24             perimetro = comprimento * 2 + largura * 2;
25             //Exibição de valores em tela
26             System.out.println("Área do retângulo: "+ area);
27             System.out.println("Perimetro do retângulo: "+ perimetro);
28         } catch (NumberFormatException erro){
29             //Bloco acionado em caso de erro de conversão de tipo
30             System.out.println("Houve erro na conversão, digite apenas "
31                 + "caracteres numéricos: "+ erro.toString());
32         }
33     }
34 }
```



- JOptionPane;

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class EntradaComJOptionPane {
4     public static void main (String args[]) {
5         //declaracao de variável
6         String aux;
7         float largura, comprimento, area, perimetro;
8         //bloco de execução com tratamento de exceções
9         try{
10             //entrada de dados
11             aux = JOptionPane.showInputDialog("Entre com o comprimento: ");
12             //conversao de tipo de dado
13             comprimento = Float.parseFloat(aux);
14             //entrada de dados
15             aux = JOptionPane.showInputDialog("Entre com o largura: ");
16             //conversao de tipo de dado
17             largura = Float.parseFloat(aux);
18             //uso de operadores aritméticos
19             area = comprimento * largura;
20             perimetro = comprimento * 2 + largura * 2;
21             //Exibição de valores em tela
22             System.out.println("Área do retângulo: "+ area);
23             System.out.println("Perimetro do retângulo: "+ perimetro);
24         } catch (NumberFormatException erro){
25             //Bloco acionado em caso de erro de conversão de tipo
26             System.out.println("Houve erro na conversão, digite apenas "
27                 + "caracteres numéricos: "+ erro.toString());
28         }
29         System.exit(0);
30     }
31 }
```



- Usando a classe `BufferedReader` para entrada de dados, crie uma classe que receba o valor de um produto e a porcentagem de desconto, calcule e mostre o valor do desconto e o valor do produto com o desconto. O valor do desconto é calculado por meio da fórmula: **valor do desconto = valor do produto * percentual de desconto / 100;**
- Usando a classe `JOptionPane` para entrada de dados, elabore uma classe que receba a nota de duas provas e de um trabalho. Calcule e mostre a média e o resultado final (aprovado ou reprovado). Para calcular a média utilize a seguinte fórmula: **média = (nota da prova 1 + nota da prova 2 + nota do trabalho) / 3.** Considere que a média mínima para aprovação é 6;



02

Estruturas Condicionais e de Exceção





- As estruturas condicionais existem em todas as linguagens de programação e possibilitam que a execução de um programa seja desviada de acordo com certas condições;
- Os comandos condicionais (ou ainda instruções condicionais) usados em Java são *if-else* e *switch-case*;
- Essas duas estruturas de desvio existentes na linguagem possibilitam executar diferentes trechos de um programa com base em certas condições;



```
if (<Condição>) {  
    <Instruções p/ condição verdadeira>  
} else {  
    <Instruções p/ condição falsa>  
}
```

```
if (<Condição>) {  
    <Instruções>  
} else if (<Condição>) {  
    <Instruções >  
} else if (<Condição>) {  
    <Instruções >  
} else {  
    <Instruções >  
}
```



- *if sem else;*

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class IfSemElse {
4     public static void main (String args[]) {
5         //declaracao de variável
6         Object[] op = {"Masculino","Feminino"};
7         //entrada de dados
8         String resp = (String) JOptionPane.showInputDialog(null,
9             "Selecione o sexo: \n", "Pesquisa",
10             JOptionPane.PLAIN_MESSAGE,
11             null, op, "Masculino");
12         //Testes condicionais
13         if (resp == "null"){
14             JOptionPane.showMessageDialog(null, "Você pressionou Cancel");
15         }
16         if (resp == "Masculino"){
17             JOptionPane.showMessageDialog(null, "Você selecionou Masculino");
18         }
19         if (resp == "Feminino"){
20             JOptionPane.showMessageDialog(null, "Você selecionou Feminino");
21         }
22         System.exit(0);
23     }
24 }
```



- *if com else;*

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class IfComElse {
4     public static void main (String args[]) {
5         //entrada de dados
6         String aux = JOptionPane.showInputDialog("Forneça o número do mês");
7         //Testes condicionais
8         if (aux != null){
9             try{
10                 int mes = Integer.parseInt(aux);
11                 if (mes >= 1 && mes <= 12){
12                     JOptionPane.showMessageDialog(null, "Número de mês válido! \n" + mes);
13                 } else {
14                     JOptionPane.showMessageDialog(null, "Número de mês inválido! \n" + mes);
15                 }
16             } catch (NumberFormatException erro){
17                 JOptionPane.showMessageDialog(null, "Digite apenas valores inteiros: "+erro);
18             }
19         } else {
20             JOptionPane.showMessageDialog(null, "Operação Cancelada.");
21         }
22         System.exit(0);
23     }
24 }
```



- *if* com *else* aninhado;

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class IfComElseAninhado {
4     public static void main (String args[]) {
5         //entrada de dados
6         String aux = JOptionPane.showInputDialog("Forneça o número do mês");
7         //Testes condicionais
8         if (aux != null){
9             try{
10                 int mes = Integer.parseInt(aux);
11                 if (mes == 1){
12                     aux = "Janeiro";
13                 } else if (mes == 2){
14                     aux = "Fevereiro";
15                 } else if (mes == 3){
16                     aux = "Março";
17                 } //inserir todos os outros meses
18                 } else if (mes == 12){
19                     aux = "Dezembro";
20                 } else {
21                     aux = "Mês Desconhecido";
22                 }
23                 JOptionPane.showMessageDialog(null, aux);
24             } catch (NumberFormatException erro){
25                 JOptionPane.showMessageDialog(null, "Digite apenas valores inteiros: "+erro);
26             }
27         } else {
28             JOptionPane.showMessageDialog(null, "Operação Cancelada.");
29         }
30         System.exit(0);
31     }
32 }
```



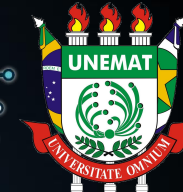
- *if* resumido;

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class IfResumido {
4     public static void main (String args[]) {
5         //declaração de variáveis
6         int a = 10, b = 15, maior;
7         //Teste condicional com if comum
8         if (a > b){
9             maior = a;
10        } else {
11            maior = b;
12        }
13        JOptionPane.showMessageDialog(null, "Usando um if comum: " + maior);
14
15        //declaração de variáveis
16        int c = 10, d = 8;
17        //Teste condicional com if comum
18        maior = (c > d) ? c : d;
19        JOptionPane.showMessageDialog(null, "Usando um if resumido: " + maior);
20    }
21 }
```



- A estrutura ***switch-case*** equivale a um conjunto de instruções ***if*** encadeadas, fornecendo maior inteligibilidade. Sua sintaxe é a seguinte:

```
switch (<expressão>){  
    case 1: instruções; break;  
    case 2: instruções; break;  
    case 3: instruções; break;  
    default: instruções;  
}
```



- *switch-case;*

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class SwitchCase {
4     public static void main (String args[]) {
5         //entrada de dados
6         String mes = (String) JOptionPane.showInputDialog("Forneça o número do mês");
7         //Testes condicionais
8         if (mes != null){
9             switch (mes){
10                 case "1":
11                     mes = "Janeiro";
12                     break;
13                 case "2":
14                     mes = "Fevereiro";
15                     break;
16                 case "3":
17                     mes = "Março";
18                     break;
19                 //inserir todos os outros meses
20                 case "12":
21                     mes = "Dezembro";
22                     break;
23                 default:
24                     mes = "Mês desconhecido";
25             }
26             JOptionPane.showMessageDialog(null, mes);
27         }
28         System.exit(0);
29     }
30 }
```



- As exceções em Java referem-se aos erros que podem ser gerados durante a execução de um programa;
- **Unchecked** (não verificadas): o código-fonte não é verificado para determinar se a exceção está sendo capturada. Assim, o tratamento desse tipo de exceção é opcional;
- **Checked** (verificadas): o código-fonte é verificado para determinar se a exceção está sendo capturada. Se uma exceção verificada não for capturada, o compilador acusa a possível exceção e obriga o programador a tratá-la. Essa exceção pode ser tratada de duas maneiras: por meio da estrutura ***try-catch-finally*** ou ***throws***;



- A linha 10 apresenta uma exceção não verificada;
- Outros exemplos de exceção desse tipo são: o acesso a um índice inexistente num *array* e o uso do método de um objeto que ainda não foi instanciado (nulo);

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class SwitchCase {
4     public static void main (String args[]) {
5         //entrada de dados
6         String mes = (String) JOptionPane.showInputDialog("Forneça o número do mês");
7         //Testes condicionais
8         if (mes != null){
9             switch (mes){
10                 case "1":
11                     mes = "Janeiro";
12                     break;
13                 case "2":
14                     mes = "Fevereiro";
15                     break;
16                 case "3":
17                     mes = "Março";
18                     break;
19                 //inserir todos os outros meses
20                 case "12":
21                     mes = "Dezembro";
22                     break;
23                 default:
24                     mes = "Mês desconhecido";
25             }
26             JOptionPane.showMessageDialog(null, mes);
27         }
28         System.exit(0);
29     }
30 }
```



- A estrutura *try-catch* tem como objetivo a previsão de erros de execução. Para exemplificar, vamos analisar a analogia abaixo:
- Considere que você vai fazer uma viagem de automóvel de uma cidade para outra. Durante o trajeto, podem ocorrer eventos que o obriguem a fazer uma pausa ou mudar seu trajeto;
- Por exemplo, ao furar o pneu do veículo você terá que parar a viagem para realizar sua troca;



- Se houver algum acidente na estrada, talvez você resolva pegar outro caminho;
- Veja que podem ocorrer vários incidentes que exigirão um tratamento diferente ou a mudança de rota;
- Algo bastante semelhante pode ocorrer durante a execução de um programa de computador: cada tipo de erro necessita que seja realizado um tratamento diferente;



- Portanto, a estrutura *try-catch-finally* tem como função desviar a execução de um programa caso ocorram certos tipos de erro;
- Quando um erro ocorre, ele gera uma exceção que pode ser tratada pelo programa. A estrutura *try-catch-finally* pode ser usada tanto com *Unchecked Exceptions* como com *Checked Exceptions*;
- Existem muitas exceções que podem ser geradas pelas mais diversas classes, e enumerá-las seria um processo dispendioso e desnecessário, pois as exceções que devem ser obrigatoriamente tratadas (*Checked Exceptions*) são apontadas pelo próprio compilador;



- A estrutura *try-catch-finally* possui a seguinte sintaxe:

```
try {  
    <conjunto de instruções>  
} catch (Nome da exceção) {  
    <tratamento do erro 1>  
} catch (Nome da exceção) {  
    <tratamento do erro 2>  
} catch (Nome da exceção) {  
    <tratamento do erro n>  
} finally {  
    <conjunto de instruções>  
}
```



- Toda vez que a estrutura *try* é utilizada, obrigatoriamente em seu encerramento (na chave final) deve existir pelo menos um *catch*, a não ser que ela utilize a instrução *finally*;
- A sintaxe apresentada pode ser interpretada como: tente executar o conjunto de instruções do *try* que estão entre as chaves; se houver algum erro, execute seu tratamento no *catch*. Depois de tratado o erro, a execução do programa continua a partir do final do último *catch*;
- O *finally* é opcional e fornece um conjunto de códigos que é sempre executado, independentemente de uma exceção ocorrer ou não;



- Um exemplo prático de sua utilização é a manipulação de arquivos de texto, em que o *bloco **finally*** será utilizado para fechar um arquivo após abri-lo, ou seja, liberar um recurso externo depois de adquiri-lo;
- Durante a execução de um programa, caso não ocorra nenhuma exceção, os blocos ***catch*** não são executados e o controle prossegue para o bloco ***finally***, que libera o recurso e passar a executar a primeira instrução após o bloco ***finally***. Caso ocorra alguma exceção, a execução do programa é desviada para o grupo de instruções ***catch*** antes de passar pelo ***finally***;



- Os erros gerados pelo programa sempre estão relacionados com as classes usadas;
- Por exemplo, o método *parseFloat* da classe *Float* (*Float.parseFloat*) pode gerar um erro de conversão numérica (*NumberFormatException*), isto é, esse tipo de erro pode ser tratado quando essa classe for usada;
- Não faz sentido verificar se ocorrerá um erro de entrada ou saída de dados (*IOException*) quando a classe *Float* for utilizada. Portanto, o tratamento de erros deve ser realizado de forma coerente, de acordo com as classes utilizadas no programa;

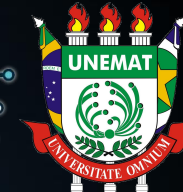


- Os erros gerados pelo programa sempre estão relacionados com as classes usadas;
- Por exemplo, o método *parseFloat* da classe *Float* (*Float.parseFloat*) pode gerar um erro de conversão numérica (*NumberFormatException*), isto é, esse tipo de erro pode ser tratado quando essa classe for usada;
- Não faz sentido verificar se ocorrerá um erro de entrada ou saída de dados (*IOException*) quando a classe *Float* for utilizada. Portanto, o tratamento de erros deve ser realizado de forma coerente, de acordo com as classes utilizadas no programa;



- **try-catch;**

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class TryCatch {
4     public static void main (String args[]) {
5         try {
6             //entrada de dados
7             String aux1 = JOptionPane.showInputDialog("Forneça o valor do 1º número:");
8             int num1 = Integer.parseInt(aux1.toString());
9             String aux2 = JOptionPane.showInputDialog("Forneça o valor do 2º número:");
10            int num2 = Integer.parseInt(aux2.toString());
11            JOptionPane.showMessageDialog(null, "Soma = " + (num1 + num2));
12            JOptionPane.showMessageDialog(null, "Subtração = " + (num1 - num2));
13            JOptionPane.showMessageDialog(null, "Multiplicação = " + (num1 * num2));
14            JOptionPane.showMessageDialog(null, "Divisão = " + (num1 / num2));
15        } catch (ArithmeticException erro) {
16            JOptionPane.showMessageDialog(null, "Erro de divisão por zero!\n" + erro.toString(),
17                "Erro", JOptionPane.ERROR_MESSAGE);
18        } catch (NumberFormatException erro) {
19            JOptionPane.showMessageDialog(null, "Erro de conversão!\n" + erro.toString(),
20                "Erro", JOptionPane.ERROR_MESSAGE);
21        } catch (NullPointerException erro) {
22            JOptionPane.showMessageDialog(null, "Tecla Cancel pressionada\n" + erro.toString(),
23                "Cancelado pelo usuário", JOptionPane.ERROR_MESSAGE);
24        }
25        System.exit(0);
26    }
27 }
```



- *try-catch-finally;*

```
1 //import da biblioteca JOptionPane
2 import javax.swing.JOptionPane;
3 class TryCatchFinally {
4     public static void main (String args[]) {
5         try {
6             //entrada de dados
7             String aux1 = JOptionPane.showInputDialog("Forneça o valor do 1º número:");
8             int num1 = Integer.parseInt(aux1.toString());
9             String aux2 = JOptionPane.showInputDialog("Forneça o valor do 2º número:");
10            int num2 = Integer.parseInt(aux1.toString());
11            JOptionPane.showMessageDialog(null, "Soma = " + (num1 + num2));
12            JOptionPane.showMessageDialog(null, "Subtração = " + (num1 - num2));
13            JOptionPane.showMessageDialog(null, "Multiplicação = " + (num1 * num2));
14            JOptionPane.showMessageDialog(null, "Divisão = " + (num1 / num2));
15        } catch (ArithmeticException erro) {
16            JOptionPane.showMessageDialog(null, "Erro de divisão por zero!\n" + erro.toString(),
17                "Erro", JOptionPane.ERROR_MESSAGE);
18        } catch (NumberFormatException erro) {
19            JOptionPane.showMessageDialog(null, "Erro de conversão!\n" + erro.toString(),
20                "Erro", JOptionPane.ERROR_MESSAGE);
21        } catch (NullPointerException erro) {
22            JOptionPane.showMessageDialog(null, "Tecla Cancel pressionada\n" + erro.toString(),
23                "Cancelado pelo usuário", JOptionPane.ERROR_MESSAGE);
24        } finally {
25            JOptionPane.showMessageDialog(null, "Final da execução!");
26        }
27        System.exit(0);
28    }
29 }
```

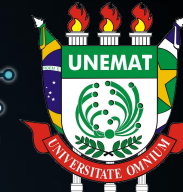


- Em alguns momentos, pode ocorrer de o programador não querer realizar controle sobre uma exceção, isto é, não desejar tratar um erro;
- A linguagem Java permite ao programador que um erro seja descartado, mesmo que ele ocorra. Entretanto é preciso que esse fato seja informado na declaração do método. Esse processo pode ser realizado pela cláusula *throws*;
- Para ilustrar o uso dessa cláusula, vamos elaborar um exemplo que demonstra a criação de um arquivo no disco rígido;



- *throws*;

```
1 //import da bibliotecas
2 import javax.swing.JOptionPane;
3 import java.io.*;
4 class UsoThrows {
5     public static void main (String args[]) throws IOException {
6         String frase = JOptionPane.showInputDialog("Entre com uma frase:");
7         try {
8             FileWriter file = new FileWriter("c:/temp/frases.txt", true);
9             PrintWriter out = new PrintWriter(file);
10            out.println(frase);
11            file.close();
12            out.close();
13            JOptionPane.showMessageDialog(null, "Frase armazenada no arquivo");
14        } catch (FileNotFoundException erro) {
15            JOptionPane.showMessageDialog(null, "Erro, verifique se a pasta c:/temp existe!");
16        }
17    }
18 }
```



- A instrução ***throw*** é utilizada para disparar uma exceção, isto é, ela pode forçar que uma determinada exceção ocorra;

```
1 //import da bibliotecas
2 import javax.swing.JOptionPane;
3 import java.io.IOException;
4 class UsoInstrucaoThrows {
5     public static void main (String args[]) throws IOException {
6         try {
7             int idade = Integer.parseInt(JOptionPane.showInputDialog("Forneça sua idade:"));
8             if (idade < 18){
9                 throw new Exception("Menor de idade, entrada não permitida!");
10            } else {
11                JOptionPane.showMessageDialog(null, "Idade válida, seja bem vindo(a)!");
12            }
13            //continua a execução normalmente
14        } catch (Exception erro) {
15            JOptionPane.showMessageDialog(null, erro.toString());
16        }
17    }
18 }
```



- Crie uma classe chamada Calculadora que solicite dois números e apresente o seguinte menu: soma, subtração, multiplicação e divisão.
- Utilize:
 - ✓ **switch-case**: para selecionar a operação;
 - ✓ **try-catch**: para tratar possíveis erros durante a execução;
 - Entrada inválida, Divisão por zero, Opção inexistente e Finalização.
 - ✓ **if/else**: para verificar situações que não devem ser permitidas.



“A dúvida é o princípio da sabedoria.”

—Aristóteles